BANORTE

MANUAL DE INSTALACIÓN

PODMAN, BUILDAH & SKOPEO

PARA USO EXCLUSIVO DE

BANORTE SEGUROS Y PENSIONES

ELABORADO POR:

ARQUITECTURA TI

BANORTE

PODMAN, BUILDAH & SKOPEO

Contenido

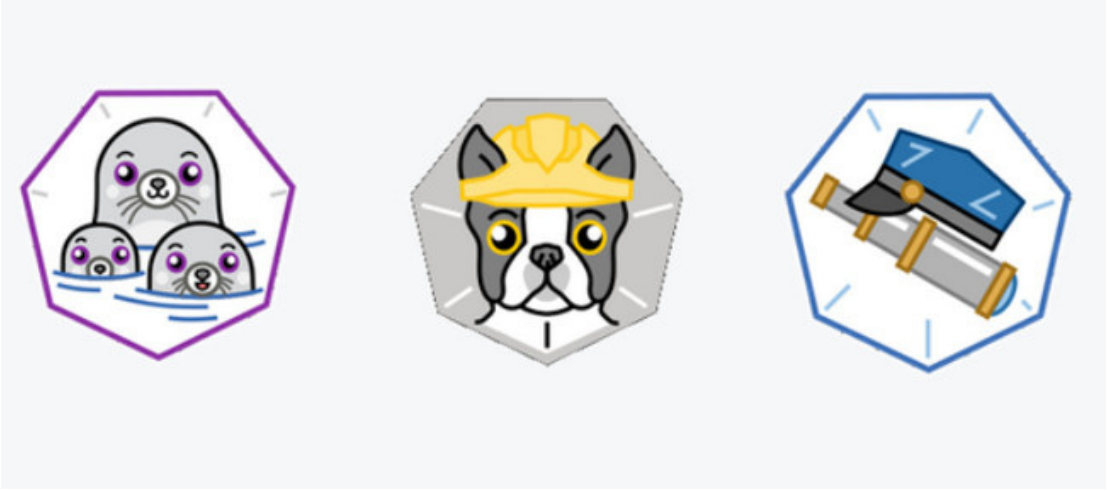
1. Introducción..... 3
2. ¿Qué es Podman, Buildah y Skopeo?..... 4
 2.1. Podman..... 5
 2.2. Buildah..... 6
 2.3. Skopeo..... 8
3. Instalaciones..... 9
 3.1. Instalación de WSL para Windows 10..... 10
 3.2. Instalación de CentOS stream sobre WSL 2..... 14
 3.3. Instalación de Podman..... 19
 3.4. Instalación de Buildah..... 20
 3.5. Instalación de Skopeo..... 21
4. Podman: comandos básicos..... 22
5. Buildah: comandos básicos..... 28
6. Skopeo: comandos básicos..... 33
7. Glosario de términos..... 37

Introducción

El presente manual tiene como finalidad permitirle utilizar herramientas como Podman, Buildah y Skopeo en un entorno Linux directamente desde su sistema operativo Windows 10, sin tener que virtualizar ni montar infraestructuras complicadas.

La instalación del Bash Linux sobre Windows es un proceso muy rápido y sencillo que apenas requiere de configuraciones. Es una instalación muy segura ya que, aunque se puede acceder al sistema de archivos de Windows e interactuar con el sistema operativo de Microsoft, nada de lo que se haga sobre esta distribución podrá hacer que Windows deje de funcionar. Como solo se carga el Kernel, es una implementación muy ligera que apenas influye en el rendimiento general del PC, esto es posible gracias a la virtualización de Hyper-V de Microsoft, y al hacerlo vamos a poder tener acceso a todos los comandos y programas de terminal de este sistema operativo directamente desde una ventana de CMD.

¿Qué es Podman, Buildah y Skopeo?



Podman, Buildah y Skopeo es la alternativa que ofrece Red Hat Enterprise Linux a Docker. Buildah construye, Podman ejecuta y Skopeo transfiere imágenes de contenedores, cabe resaltar que estas herramientas son de código abierto y en su mayor parte, tampoco necesitan acceso root, salvo algunas excepciones.

Podman, Buildah y Skopeo son herramientas Linux por lo que solo funcionan dentro de las principales distribuciones como Ubuntu, Fedora, CentOS, Debian y RHEL, por esta razón, para trabajar dentro del sistema operativo de Microsoft es necesario contar con la instalación de una máquina virtual o en nuestro caso hacer una instalación previa de Windows Subsystem for Linux para poder montar dicha distribución.

Podman

Podman puede ejecutar y administrar contenedores e imágenes de contenedores. Tiene características avanzadas, como el soporte para ejecutar contenedores en Pods, lo que facilita la migración posterior de las cargas de trabajo a Kubernetes.

Los contenedores bajo el control de Podman pueden ser ejecutados por root o por un usuario sin privilegios y al ser daemon-less no requiere levantar un proceso de muchos servicios para funcionar, ya que levanta los servicios necesarios para cada contenedor, haciendo más tratable la gestión de servicios en el anfitrión.

Podman se basa en un Container Runtime compatible con Open Container Initiatives (OCI) para interactuar con el sistema operativo y crear los contenedores en ejecución. Esto le permite a Podman proporcionar los mismos comandos CLI que Docker ya que se implementan utilizando el mismo estándar. De hecho si tiene scripts existentes que ejecutan Docker, puede establecer un alias que le permita el funcionamiento de dichos scripts (alias docker=podman), así cambiar de Docker a Podman suele ser relativamente sencillo.

Podman puede empujar y extraer de registros de contenedores populares como Quay.io y Docker hub, así como registros privados.

podman

Buildah

Buildah tiene la capacidad de crear imágenes de contenedor desde cero, pero también desde un Dockerfile preexistente. Estas imágenes se pueden enviar a cualquier registro de contenedores y pueden ser usadas por cualquier motor de contenedores, incluidos Podman, CRI-O y Docker.

Buildah, se puede describir como un superconjunto de comandos relacionados con la creación y administración de imágenes de contenedor y, por lo tanto, tiene un control mucho más detallado sobre las imágenes. Sin asignación de puertos, sin montaje de volumen, ya que esas banderas fueron retiradas.

Buildah le permite tener un clúster de Kubernetes sin ningún Docker Daemon, tanto para el tiempo de ejecución como para las compilaciones. Pero, ¿que sucede si las cosas salen mal y desea solucionar esos problemas o depurar contenedores en su clúster? Buildah no esta diseñado realmente para eso, lo que necesita es una herramienta de cliente para trabajar con contenedores y podría pensar en Docker CLI, pero entonces volvería a utilizar Docker Daemon. Aquí es donde interviene Podman, ya que este permite ejecutar, compilar, modificar y solucionar problemas de contenedores en su clúster de Kubernetes sin la dependecia Daemon.

Algunos de los comandos entre Buildah y Podman se superponen, pero en algunos casos tienen comportamientos ligeramente diferentes. Por ejemplo, **buildah run** emula el comando RUN de un Dockerfile, mientras que **podman run** emula el comando docker run en funcionalidad.

En resumen, Buildah es una forma eficiente de crear imágenes OCI, mientras que Podman le permite administrar y mantener esas imágenes y contenedores en un entorno de producción utilizando comandos de CLI. Juntos forman una base sólida para respaldar su imagen de contenedor OCI y sus necesidades de contenedor.

buildah

Skopeo

Skopeo es una utilidad de línea de comandos que realiza varias operaciones en imágenes de contenedor y repositorios de imágenes. Puede trabajar con imágenes OCI, así como las imágenes originales de Docker v2.

Skopeo trabaja con registros y no requiere de un Daemon que se ejecute para realizar operaciones que consisten en:

- Copiar una imagen desde y hacia varios mecanismos de almacenamiento. Por ejemplo, puede copiar imágenes de un registro a otro, sin necesidad de privilegios.
- Inspeccionar una imagen remota que muestra sus propiedades, incluidas sus capas, sin necesidad de extraer la imagen al host.
- Eliminación de una imagen de un repositorio de imágenes.
- Cuando el repositorio lo requiera, skopeo puede pasar las credenciales y certificados apropiados para la autenticación.

skopeo

Instalaciones

Windows Subsystem for Linux también denominado WSL, es una capa que implementa Microsoft en los sistemas operativos Windows 10 y Windows server 2019 para proporcionar compatibilidad y ejecución de ejecutables binarios Linux, sin ningún uso de software de Máquinas virtuales como VirtualBox.

Para instalar una imagen de Linux, dependemos de la tienda de Microsoft donde están presentes ciertas imágenes ligeras precompiladas. Actualmente las imágenes WSL de Linux disponibles son Ubuntu, Kali Linux, Debian, OpenSUSE y SUSE. Por lo tanto, si se desea probar algunos sistemas operativos Linux que no estén dentro de la lista, entonces tenemos que instalarlos manualmente.

Nota: Para trabajar con Podman, Buildah y Skopeo montaremos una imagen CentOS-stream, pero antes de hacerlo se debe haber habilitado el subsistema de Windows para Linux, si ya cuenta con WSL en su sistema operativo Windows 10 favor de pasar directamente a la Instalación de CentOS-stream.

Instalación de WSL para Windows 10

Hay dos opciones para instalar el Subsistema de Windows para Linux, la instalación simplificada y la instalación manual, para este fin utilizaremos la instalación manual ya que para la primera deberemos contar con un registro al programa Windows Insider, pero si se requiere la instalación simplificada puede hacer clic en el siguiente enlace:

Instalación de WSL en Windows 10 | Microsoft Docs

Instalación Manual

1. Habilitación del Subsistema de Windows para Linux

Antes de instalar distribuciones de Linux en Windows, se debe habilitar la característica "Subsistema de Windows para Linux" WSL.

Abrir PowerShell como administrador y ejecutar.

```
dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /norestart
```

2. Comprobar los requisitos para ejecutar WSL 2

Para actualizar a WSL 2, compruebe la versión y el número de compilación seleccionando la tecla Windows + R, escriba winver y seleccione aceptar.

- Para sistemas x64: La versión 1903 o posterior, con la compilación 18362 o posterior.
- Para sistemas ARM64: La versión 2004 o posterior, con la compilación 19041 o posterior.
- Las compilaciones anteriores a 18362 no admiten WSL 2. Use el Asistente para Windows Update para actualizar su versión de Windows.

Si está ejecutando Windows 10, versión 1903 o 1909, abra "Configuración" en el menú de Windows, vaya a "Actualización y seguridad" y seleccione "Buscar actualizaciones". El número de compilación debe ser 18362.1049 o posterior o 18363.1049 o posterior, con la compilación secundaria posterior a .1049.

3. Habilitación de la característica Máquina Virtual

Antes de instalar WSL 2, se debe habilitar la característica Plataforma de máquina virtual, ya que se necesitarán funcionalidades de virtualización para el uso de WSL 2.

Abrir PowerShell como administrador y ejecutar.

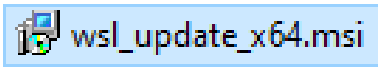
```
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart
```

Reinicia tu equipo para completar la instalación de WSL y la actualización de WSL 2.

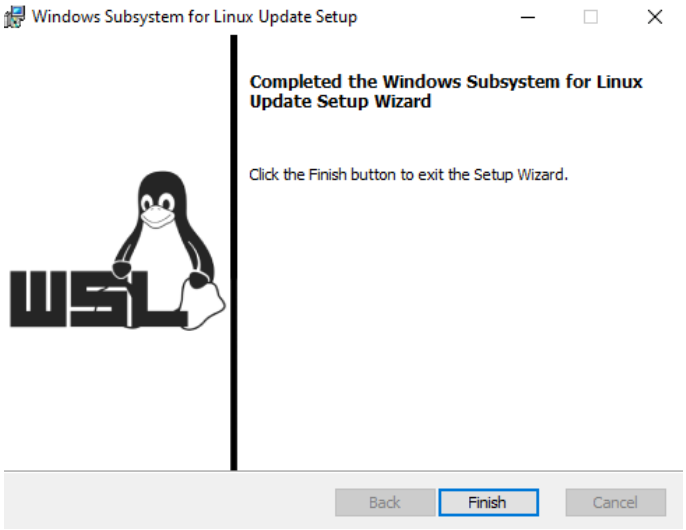
4. Descarga y ejecuta el paquete de actualización del Kernel de Linux

a) Descargue la versión más reciente:

Paquete de actualización del kernel de Linux en WSL 2 para máquinas x64



b) Ejecuta el paquete de actualización que descargaste en el paso anterior. (Haga doble clic para ejecutarlo. Se solicitarán permisos elevados. Seleccione sí para aprobar esta instalación).



5. Definir WSL 2 como versión predeterminada

Abrir PowerShell y ejecutar el comando para establecer WSL 2 como versión predeterminada al instalar una nueva distribución de Linux

```
wsl --set-default-version 2
```

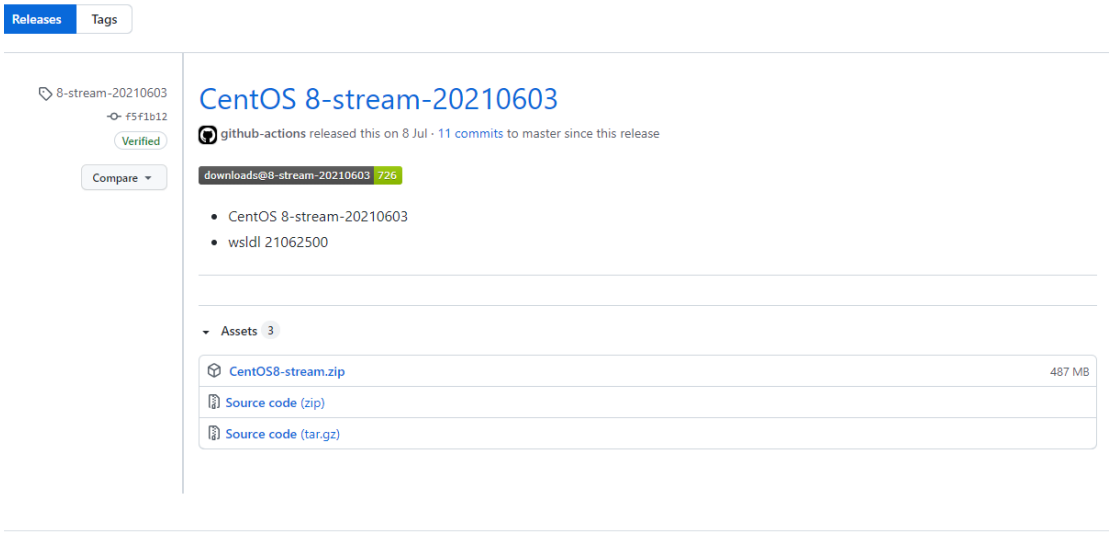
Instalar CentOS-stream sobre WSL

1. Descargar CentOS-stream WSL

Lo primero que necesitamos es CentOS-stream para WSL, básicamente es un archivo que contiene rootfs y otros archivos. CentOS-stream como instancia WSL es un proyecto de código abierto disponible en Github, que se puede conseguir desde aquí.

[Release CentOS 8-stream-20210603 · mishamosher/CentOS-WSL · GitHub]

Una vez en el repositorio, daremos clic sobre la carpeta **CentOS8-stream.zip** para descargar.



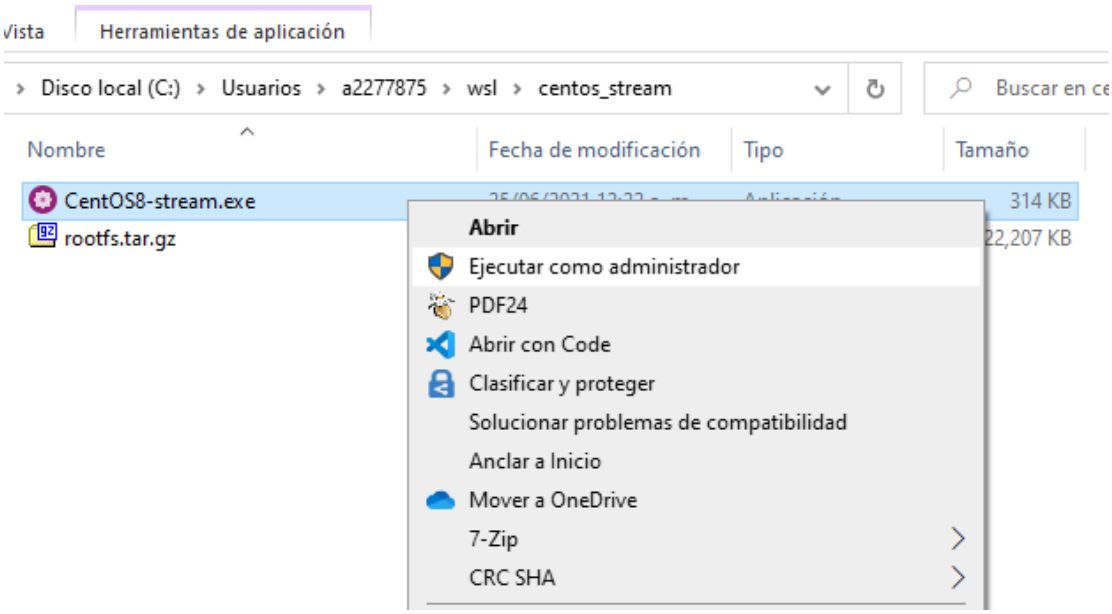
2. Descomprima la carpeta zip

En la carpeta raíz crearemos un directorio llamado wsl y dentro crearemos otra carpeta con el nombre de la distribución que vamos a descomprimir.

wsl > centos-stream

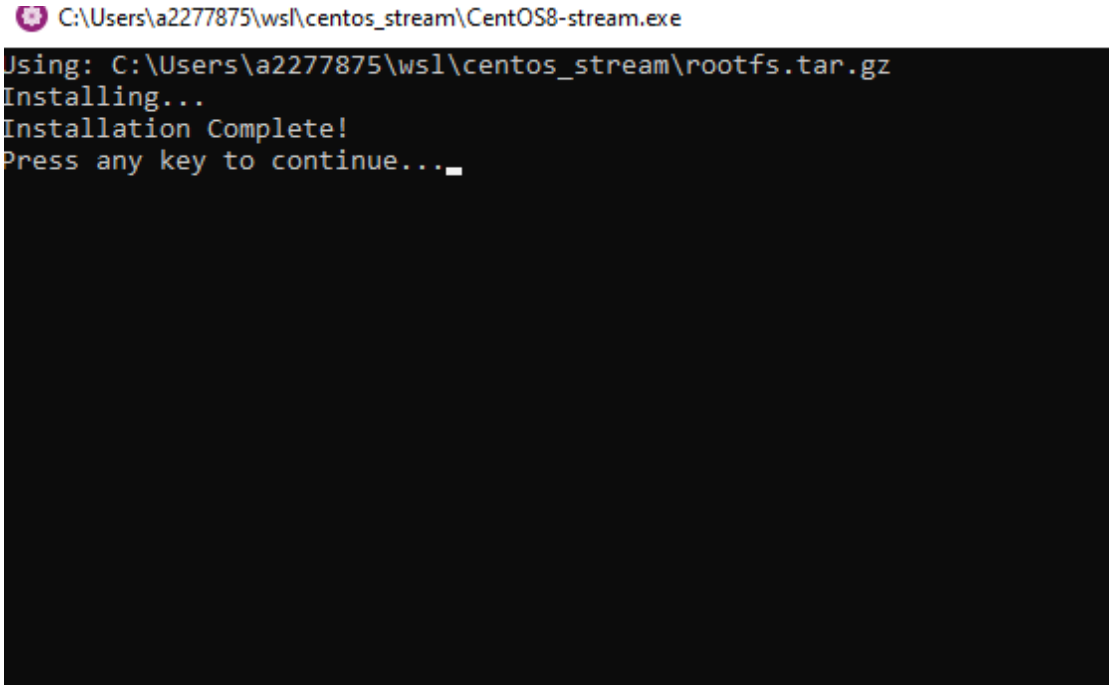
3. Ejecute CentOS8-stream para instalarlo en WSL

Después de extraer la carpeta, visualizara dos archivos uno llamado rootfs.tar.gz y el segundo es un ejecutable de CentOS8-stream. Necesitamos ejecutar CentOS8-stream para extraer los archivos y registrarlos en WSL. Damos clic derecho y ejecutamos como administrador.



4. Instalación completada

Una vez que se complete la instalación, recibirá un mensaje en la pantalla de comandos de CentOS-stream. Presione la tecla Enter para salir de la configuración.



5. Ejecute CentOS8-stream para comenzar a usar

Para iniciar nuestra distribución regrese a la carpeta extraída y haga clic sobre el ejecutable.

Finalmente, nuestro subsistema Linux instalado manualmente está aquí.

6. Actualizar CentOS-stream

Puede ejecutar cualquier comando propio de CentOS-stream incluso personalizar la terminal con zsh, sin embargo, solo para verificar si la instalación fue correcta ejecutaremos el comando para actualizar la distribución.

```
yum -y update
```

```
root@CON-A2277875L:~
root@CON-A2277875L ~]# yum -y update
entOS Stream 8 - AppStream                                      2.1 MB/s |  14 MB     00:06
entOS Stream 8 - BaseOS                                         2.1 MB/s |  12 MB     00:05
entOS Stream 8 - Extras                                          12 kB/s |  15 kB     00:01
ependencies resolved.
================================================================================================================
 Package                          Architecture Version                                    Repository       Size
================================================================================================================
nstalling:
 kernel                           x86_64       4.18.0-338.el8                             baseos          7.0 M
 kernel-core                      x86_64       4.18.0-338.el8                             baseos           38 M
 kernel-modules                   x86_64       4.18.0-338.el8                             baseos           30 M
pgrading:
 NetworkManager                   x86_64       1:1.32.10-2.el8                            baseos          2.6 M
 NetworkManager-libnm             x86_64       1:1.32.10-2.el8                            baseos          1.8 M
 NetworkManager-team              x86_64       1:1.32.10-2.el8                            baseos          148 k
 NetworkManager-tui               x86_64       1:1.32.10-2.el8                            baseos          336 k
 authselect                       x86_64       1.2.2-3.el8                                baseos          133 k
 authselect-compat                x86_64       1.2.2-3.el8                                appstream        38 k
 authselect-libs                  x86_64       1.2.2-3.el8                                baseos          222 k
 bash                             x86_64       4.4.20-2.el8                               baseos          1.5 M
 bind-export-libs                 x86_64       32:9.11.26-6.el8                           baseos          1.1 M
 ca-certificates                  noarch       2021.2.50-82.el8                           baseos          390 k
 centos-gpg-keys                  noarch       1:8-3.el8                                  baseos           12 k
 centos-logos                     x86_64       85.8-1.el8                                 appstream       700 k
 centos-stream-release            noarch       8.6-1.el8                                  baseos           22 k
 centos-stream-repos              noarch       8-3.el8                                    baseos           19 k
 chkconfig                        x86_64       1.19.1-1.el8                               baseos          198 k
 chrony                           x86_64       4.1-1.el8                                  baseos          327 k
 cloud-init                       noarch       21.1-7.el8                                 appstream       1.0 M
```

Nota: Si por alguna razón requiere desinstalar su distribución y reinstalarla primero sera necesario anular el registro de la siguiente manera:

Abra Powershell y verifique como se generó el nombre de la distribución que anularemos enlistando las distribuciones instaladas con el comado:

```
wsl --list --verbose
```

Windows PowerShell

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Prueba la nueva tecnología PowerShell multiplataforma https://aka.ms/pscore6

PS C:\Users\a2277875> wsl -l -v
  NAME                   STATE           VERSION
* docker-desktop         Stopped         2
  CentOS8-stream         Stopped         2
  Ubuntu-20.04           Stopped         2
  docker-desktop-data    Stopped         2
PS C:\Users\a2277875>
```

A continuación escriba el comando --unregister mas el nombre de la distribución para anular el registro

```
wsl --unregister CentOS8-stream
```

Cuando haya terminado, puede proceder a reinstalar y configurar la distribución si así lo desea.

Instalación de Podman

1. Ejecutar comando de instalación

Abra la terminal de CentOS stream y ejecute el comando

```
sudo dnf -y install podman
```

```
/
# sudo dnf -y install podman                                                    root@CON-A2277875L
Last metadata expiration check: 5:15:06 ago on Tue 05 Oct 2021 10:58:24 AM CDT.
Dependencies resolved.
================================================================================================================
 Package                          Architecture  Version                                        Repository      Size
================================================================================================================
Installing:
 podman                           x86_64        3.3.1-9.module_el8.6.0+938+04eb0c17            appstream       12 M
Installing dependencies:
 conmon                           x86_64        2:2.0.29-1.module_el8.6.0+926+8bef8ae7         appstream       52 k
 container-selinux                noarch        2:2.167.0-1.module_el8.6.0+926+8bef8ae7        appstream       54 k
 containernetworking-plugins      x86_64        1.0.0-1.module_el8.6.0+926+8bef8ae7            appstream       19 M
 containers-common                noarch        2:1-2.module_el8.6.0+926+8bef8ae7              appstream       79 k
 criu                             x86_64        3.15-3.module_el8.6.0+926+8bef8ae7             appstream      518 k
 fuse-common                      x86_64        3.2.1-12.el8                                   baseos          21 k
 fuse-overlayfs                   x86_64        1.7.1-1.module_el8.6.0+926+8bef8ae7            appstream       73 k
 fuse3                            x86_64        3.2.1-12.el8                                   baseos          50 k
 fuse3-libs                       x86_64        3.2.1-12.el8                                   baseos          94 k
 iptables                         x86_64        1.8.4-20.el8                                   baseos         585 k
 libnet                           x86_64        1.1.6-15.el8                                   appstream       67 k
 libnetfilter_conntrack           x86_64        1.0.6-5.el8                                    baseos          65 k
 libnfnetlink                     x86_64        1.0.1-13.el8                                   baseos          33 k
 libnftnl                         x86_64        1.1.5-4.el8                                    baseos          83 k
 libslirp                         x86_64        4.4.0-1.module_el8.6.0+926+8bef8ae7            appstream       70 k
 nftables                         x86_64        1:0.9.3-21.el8                                 baseos         321 k
 podman-catatonit                 x86_64        3.3.1-9.module_el8.6.0+938+04eb0c17            appstream      340 k
 protobuf-c                       x86_64        1.3.0-6.el8                                    appstream       37 k
 runc                             x86_64        1.0.2-1.module_el8.6.0+926+8bef8ae7            appstream      3.1 M
 slirp4netns                      x86_64        1.1.8-1.module_el8.6.0+926+8bef8ae7            appstream       51 k
```

Para corroborar la instalación ejecute el comando

```
podman version
```

```
/ # podman version                                                            root@CON-A2277875L
Version:      3.3.1
API Version:  3.3.1
Go Version:   go1.16.7
Built:        Wed Sep 22 10:22:49 2021
OS/Arch:      linux/amd64
```

Instalación de Buildah

1. Ejecutar comando de instalación

Abra la terminal de CentOS stream y ejecute el comando

```
sudo dnf -y install buildah
```

```
/ # sudo dnf -y install buildah                                                                    root@CON-A2277875L
Last metadata expiration check: 6:18:03 ago on Tue 05 Oct 2021 10:58:24 AM CDT.
Dependencies resolved.
================================================================================================================================
 Package            Architecture       Version                                          Repository            Size
================================================================================================================================
Installing:
 buildah            x86_64             1.22.3-2.module_el8.6.0+926+8bef8ae7             appstream            7.7 M

Transaction Summary
================================================================================================================================
Install  1 Package

Total download size: 7.7 M
Installed size: 28 M
Downloading Packages:
buildah-1.22.3-2.module_el8.6.0+926+8bef8ae7.x86_64.rpm                            3.0 MB/s | 7.7 MB     00:02
--------------------------------------------------------------------------------------------------------------------------------
Total                                                                              2.5 MB/s | 7.7 MB     00:03
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :                                                                                                        1/1
  Installing       : buildah-1.22.3-2.module_el8.6.0+926+8bef8ae7.x86_64                                                    1/1
  Running scriptlet: buildah-1.22.3-2.module_el8.6.0+926+8bef8ae7.x86_64                                                    1/1
  Verifying        : buildah-1.22.3-2.module_el8.6.0+926+8bef8ae7.x86_64                                                    1/1
```

Ejecute el comando info para corroborar la instalación

```
buildah version
```

```
/ # buildah version
Version:         1.22.4-dev
Go Version:      go1.16.7
Image Spec:      1.0.1-dev
Runtime Spec:    1.0.2-dev
CNI Spec:        0.4.0
libcni Version:  v0.8.1
image Version:   5.15.2
Git Commit:
Built:           Mon Sep 20 18:02:40 2021
OS/Arch:         linux/amd64
```

Instalación de Skopeo

1. Ejecutar comando de instalación

Abra la terminal de CentOS stream y ejecute el comando

```
sudo dnf -y install skopeo
```

```
~ # sudo dnf -y install skopeo                                                    root@CON-A2277875L
Last metadata expiration check: 1:01:25 ago on Tue 05 Oct 2021 07:23:37 PM CDT.
Dependencies resolved.
================================================================================================================
 Package            Architecture       Version                                        Repository            Size
================================================================================================================
Installing:
 skopeo             x86_64             1:1.4.2-0.1.module_el8.6.0+926+8bef8ae7        appstream             6.7 M

Transaction Summary
================================================================================================================
Install  1 Package

Total download size: 6.7 M
Installed size: 25 M
Downloading Packages:
skopeo-1.4.2-0.1.module_el8.6.0+926+8bef8ae7.x86_64.rpm                    2.2 MB/s | 6.7 MB     00:03
----------------------------------------------------------------------------------------------------------------
Total                                                                      1.3 MB/s | 6.7 MB     00:05
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :                                                                                        1/1
  Installing       : skopeo-1:1.4.2-0.1.module_el8.6.0+926+8bef8ae7.x86_64                                  1/1
  Running scriptlet: skopeo-1:1.4.2-0.1.module_el8.6.0+926+8bef8ae7.x86_64                                  1/1
  Verifying        : skopeo-1:1.4.2-0.1.module_el8.6.0+926+8bef8ae7.x86_64                                  1/1
```

Ejecute el comando info para corroborar la instalación

```
skopeo --version
```

```
~ # skopeo --version
skopeo version 1.4.2-dev

~ #
```

Podman: conceptos básicos

Una vez presentadas las características básicas de Podman y la instalación corriendo sobre CentOS veamos algunos comandos básicos que nos permitiran comenzar con este motor de contenedores.

Nota: Si tag se omite de image:tag, el valor por default es latest.

Encontrar imágenes

podman images
Enlista todas las imágenes locales

podman history image:tag
Muestra información sobre cómo se creó una imagen

podman login registryURL -u username [-p password]
Inicie sesión en un registro remoto

podman pull registry/username/image:tag
Extraiga una imagen de un registro remoto

podman search searchString
Buscar imágenes en caché local y registros remotos

Nota: La lista de registros esta definida en /etc/containers/registries.conf

podman logout
Cerrar sesión en el registro remoto actual

Construyendo imágenes

podman build -t image:tag .
Construya y etiquete una imagen usando las instrucciones de un Dockerfile en el directorio actual (¡no olvide el punto!)

podman build -t image:tag -f Dockerfile2
Igual que el anterior, pero con un Dockerfile diferente

podman tag image:tag image:tag2
Agrega un nombre adicional a una imagen local

podman tag image:tag registry/username/image:tag
Igual que el anterior, pero el nombre adicional incluye un registro remoto

podman push registry/username/image:tag
Enviar una imagen a un registro remoto

Ejecutando contenedores sobre imágenes

podman run --rm -it [--name name] image:tag command
Ejecute un contenedor basado en una imagen determinada

--rm	Elimine el contenedor después de existir
-it	Conectar el contenedor a la terminal
--name name	Dale un nombre al contenedor
image:tag	La imagen utilizada para crear el contenedor
command	Un comando para ejecutar (/ bin/bash por ejemplo)

Otras opciones

-d
Ejecute el contenedor en segundo plano

-p 8080:32000
Exponer el puerto de contenedor 8080 como localhost: 32000

-v /var/lib/mydb:/var/lib/db
Asigne el directorio /var/lib/mydb en localhost a un volumen llamado /var/lib/db dentro del contenedor

podman commit container newImage:tag
Crea una nueva imagen basada en el estado actual de un contenedor en ejecución

podman create [--name name] image:tag
Cree (pero no comience) un contenedor a partir de una imagen

podman start container
Iniciar un contenedor existente a partir de una imagen

podman restart container
Reiniciar un contenedor existente

podman wait container1 [container2…]
Espere a que uno o más contenedores se detengan

podman stop container
Detenga un contenedor en funcionamiento pasado unos seg.

podman kill container
Elimina uno o más contenedores en ejecución con una señal específica

podman rm [-f] container

Remueva un contenedor (use -f si el contenedor se está ejecutando)

podman stats container

Mostrar una transmisión en vivo del uso de recursos de un contenedor

podman inspect container

Devolver metadatos (en JSON) sobre un contenedor en ejecución

Trabajar con procesos y recursos de contenedores

podman ps [--all]

Enumere los contenedores en ejecución en el sistema (use --all para incluir los contenedores que no se ejecutan)

podman attach container

Adjuntar a un contenedor en ejecución y ver su salida o controlarlo Ctrl + p Ctrl + q se desprende del contenedor pero lo deja funcionando.

podman exec container command

Ejecutar un comando en un contenedor en ejecución

podman top container

Mostrar los procesos en ejecución de un contenedor

podman logs [-tail] container

Mostrar los registros de un contenedor

podman pause container | podman unpause container

Pausar / reanudar todos los procesos en un contenedor

podman port container

Enumere las asignaciones de puertos de un contenedor a localhost

Trabajar con el sistema de archivos de un contenedor

podman diff container

Mostrar todos los cambios en un contenedor

podman cp source target

Copie archivos y carpetas entre un contenedor y localhost

podman mount container | podman umount container

Montar o desmontar la raíz de un contenedor

podman import tarball

Importe un tarball (.tar, .tar.gz, .tgz, .bzip, .tar.xz, .txz) y guárdelo como una imagen del sistema de archivos

podman export [-o outputFile] container

Exportar el sistema de archivos del contenedor a un archivo tar

podman save [-o archiveFile] [--format docker-archive | oci-archive | oci-dir | docker-dir] image:tag

Guarde una imagen en docker-archive (predeterminado) u otro formato

podman load -i archiveFile

Cargue una imagen guardada desde docker-archive u otro formato

Removiendo imágenes

podman rmi [-f] image:tag
Eliminar una imagen local de la caché local (use -f para forzar la eliminación)

podman rmi [-f] registry/username/image:tag
Eliminar una imagen remota de la caché local (use -f para forzar la eliminación)

Nota: esto no elimina la imagen del registro remoto.

Otros

podman version
Mostrar información de la versión de podman

podman info
Mostrar información sobre el entorno de podman

Más Información

Para más información acerca de Podman, visite el sitio web Podman.io o Red Hat Developer.

Buildah: conceptos básicos

Ahora veamos el conjunto de comandos de construcción para trabajar con Buildah.

Consultas

buildah containers
Enumerar todos los contenedores de trabajo

buildah images
Enumerar todas las imágenes locales

buildah inspect container-id
Muestra detalles sobre un contenedor

buildah inspect --type image image-id
Muestra detalles sobre una imagen

Creando contenedores

buildah from base
Crea un nuevo contenedor de trabajo a partir de una imagen base; el ID del contenedor se devuelve de esta llamada

buildah from scratch
Crea una nueva imagen desde cero sin base; el nombre del contenedor se devuelve de esta llamada (consulte "Scratch Containers" para obtener más información)

Edición de contenedores

buildah run container-id — commands

Ejecuta los comandos después del (-) dentro del contenedor especificado (similar en concepto al comando RUN en un Dockerfile)

buildah copy container-id local-file destination

Copia un archivo al destino especificado dentro del contenedor dado

buildah config flags container-id

Modifica los valores de configuración que se guardarán en la imagen de un contenedor cuando se cree

Algunos indicadores de ejemplo son los siguientes (la lista completa de opciones de configuración se puede encontrar ejecutando buildah config -h):

--cmd command-string	Establece la ejecución del comando predeterminado para los contenedores creados a partir de esta imagen
--entrypoint entrypoint-string	Establece el punto de entrada para los contenedores creados a partir de esta imagen
--port port	Agrega un puerto para ser expuesto por los contenedores de esta imagen

buildah commit container-id image-name

Crea una nueva imagen basada en el estado actual del contenedor proporcionado

Borrando contenedores

buildah rm container-id

Elimina el contenedor dado; Se pueden especificar múltiples ID de contenedor, separados por espacios.

buildah rm --all

Eliminar todos los contenedores

Borrando imágenes

buildah rmi image-id

Elimina la imagen dada

buildah rmi --all

Elimina todas las imágenes

buildah rmi --prune

Elimina las imágenes no utilizadas del almacenamiento local

Scratch containers

Una vez que se ha inicializado un contenedor desde cero, los comandos de "Edición de contenedores" también se pueden utilizar para manipularlos. Los siguientes comandos se utilizan para establecer el conjunto de paquetes necesarios en la imagen desde cero.

buildah from scratch

crea una nueva imagen temporal sin base; el nombre del contenedor se devuelve de esta llamada

buildah mount container-id

Monta una imagen temporal en un directorio del sistema local y devuelve la ruta completa al directorio

buildah unshare buildah mount container-id

Monta una imagen temporal para usuarios que no son root

dnf install --installroot scratch-mount-dir package-list --setopt install_weak_deps=false -y

Ejemplo, llamada al administrador de paquetes dnf para instalar paquetes en el directorio en el que está montado el contenedor temporal

buildah unmount container-id

Desmontar un contenedor previamente montado

buildah commit container-id image-name

Crea una nueva imagen basada en el estado actual del contenedor proporcionado, incluidos los cambios realizados mientras se montaba el contenedor

Usando Dockerfiles

buildah bud -t image-tag

Crea una imagen de un Dockerfile en el directorio actual (bud es la abreviatura de build-using-dockerfile que también se puede usar)

Interactuar con registros

buildah login registry
Inicia sesión en el registro especificado, como docker.io o quay.io

buildah push image-tag
Envía la imagen especificada a un repositorio remoto

buildah tag existing-image-tag new-image-tag
Crear una etiqueta de imagen adicional en una imagen existente

Más información

Para más información acerca de Buildah visite el sitio web Buildah.io o Red Hat Developers.

Skopeo: conceptos básicos

Como ya se abordo anteriormente, Skopeo es una utilidad de línea de comandos que realiza varias operaciones en imagenes de contenedor y repositorios de imágenes por lo que Skopeo opera en los siguientes tipos de imágenes y repositorios:

containers-storage:docker-reference

Una imagen ubicada en una tienda de imágenes de almacenamiento / contenedores local.
Tanto la ubicación como el almacén de imágenes se especifican en /etc/containers/storage.conf. (Este es el backend para Podman, CRI-O, Buildah y amigos).

dir:path

Una ruta de directorio local existente que almacena el manifiesto, los archivos tar de la capa y las firmas como archivos individuales. Este es un formato no estandarizado, principalmente útil para depuración o inspección de contenedores no invasiva.

docker://docker-reference

Una imagen en un registro que implementa "Docker Registry HTTP API V2". De forma predeterminada, utiliza el estado de autorización en $XDG_RUNTIME_DIR/containers/auth.json, que se establece mediante skopeo login

docker-archive:path[:docker-reference]

Una imagen se almacena en un archivo docker save formateado. docker-reference solo se usa al crear un archivo de este tipo, y no debe contener un resumen

docker-daemon:docker-reference
Imagen docker-reference almacenada en el almacenamiento interno del docker daemon. Docker-reference debe contener una etiqueta o un resumen. Alternativamente, al leer imágenes, el formato también puede ser docker-daemon:algo:digest (un ID de imagen).

oci:path:tag
Una etiqueta de imagen en un directorio que cumple con la "Especificación de diseño de imagen de contenedor abierto" en la ruta.

Operación inspección

Obtiene el manifiesto del repositorio y puede mostrarle una salida json similar a la de un repositorio completo o una etiqueta.

skopeo inspect docker://docker-reference
Mostrar propiedades

skopeo inspect --config docker://docker-reference | jq
Mostrar la configuración del contenedor

skopeo inspect docker://docker-reference | jq '.Digest'
Mostrar resumen de imagen no verificada

Operación copiar

Skopeo puede copiar imágenes de contenedor entre varios mecanismos de almacenamiento.

skopeo copy docker://docker-reference docker://docker-reference
skopeo copy oci:path:tag dir:path

Registros de contenedores:

- The Quay, Docker Hub, OpenShift, GCR, Artifactory...

Backends de almacenamiento

- github.com/containers/storage (Backend para Podman, CRI-O, Buildah y amigos)
- Almacenamiento de docker daemon

Directorios Locales

Directorios de diseño de OCI locales

Operación borrar

```
skopeo delete docker://docker-reference
```

Operación sincronizar

```
skopeo sync --src docker --dest dir registry.example.com/busybox
/media/usb
```

Operación autenticación

Skopeo utiliza credenciales de las banderas --creds (para skopeo inspect|delete) o --src-creds|--dest-creds (para skopeo copy), si se establece; de lo contrario, utiliza la configuración establecida por skopeo login, podman login, buildah login o docker login.

```
skopeo login --username USER myregistrydomain.com:5000
Password:
skopeo inspect docker://myregistrydomain.com:5000/busybox
Returna registro formato json
skopeo logout myregistrydomain.com:5000
```

Usando --creds directamente

```
skopeo inspect --creds=testuser:testpassword docker://docker-reference
```

```
skopeo copy --src-creds=testuser:testpassword docker://docker-reference oci:path:tag
```

Más Información

Para más información acerca de Skopeo visite el sitio web Repositorio skopeo o access.redhat.com

Glosario de términos

Bash

Es un intérprete de comandos que ejecuta, una por una, las instrucciones introducidas por el usuario o contenidas en un script y devuelve los resultados.

Actúa como interfaz entre el kernel Linux y los usuarios o programas del modo texto. Bash no es una terminal. Tampoco es la única shell disponible. De hecho, podemos ver los intérpretes disponibles en nuestro equipo al observar el contenido del archivo /etc/shells.

Container Runtime

Es el software responsable de ejecutar los contenedores.

Contenedor

Es un formato de empaquetado ejecutable, independiente y liviano que incluye todo lo necesario para ejecutar una aplicación, incluidas bibliotecas, herramientas del sistema, código y tiempo de ejecución.

CLI

La interfaz de línea de comandos (en inglés, command-line interface, CLI) es un tipo de interfaz de usuario de computadora que permite a los usuarios dar instrucciones a algún programa informático o al sistema operativo por medio de una línea de texto simple.

CMD

Es una abreviatura que significa “Command” y utilizada en el área de la informática para referirse a una ventana de comandos denominada en el sistema de Windows como “Símbolo de sistema”.

CRI-O

Es una alternativa ligera al uso de Docker como tiempo de ejecución para kubernetes. Permite a Kubernetes usar cualquier tiempo de ejecución compatible con OCI como tiempo de ejecución de contenedor para ejecutar pods.

Daemon

Es un programa de computadora que se ejecuta como un proceso de fondo (background process), en lugar de estar en control directo de un usuario.

Distribucion Linux

Una distribución Linux (coloquialmente llamada distro) es una colección de software basada en el núcleo Linux que incluye determinados paquetes de software, por lo general están compuestas, total o mayoritariamente, de software libre, aunque a menudo incorporan aplicaciones o controladores propietarios.

Docker

Es una plataforma de software que permite a los desarrolladores crear, probar e implementar aplicaciones en contenedores rápidamente.

Docker Daemon

Es un proceso daemon con el que nos comunicamos a los contenedores de docker. Para comunicarnos con el proceso daemon utilizarmos el cliente de docker o docker client.

Dockerfile

Es un archivo de texto que contiene las instrucciones necesarias para crear una nueva imagen del contenedor. Estas instrucciones incluyen la identificación de una imagen existente que se usará como base, los comandos que se ejecutarán durante el proceso de creación de la imagen y un comando que se ejecutará cuando se implementen instancias nuevas de la imagen del contenedor.

Docker hub

Docker Hub es un servicio de registro de repositorios proporcionado por Docker Inc. Nos permite extraer y enviar imágenes de la ventana acoplable hacia y desde Docker Hub.

Hyper-V

Es el producto de virtualización de hardware de Microsoft. Permite crear y ejecutar una versión de software de un equipo, denominada máquina virtual. Cada máquina virtual actúa como un equipo completo, ejecutando un sistema operativo y programas. Cuando necesita recursos informáticos, las máquinas virtuales proporcionan más flexibilidad, ayudan a ahorrar tiempo y dinero, y son una manera más eficaz de usar hardware que simplemente ejecutar un sistema operativo en hardware físico.

Imágenes de contenedores

Es un archivo estático e inmutable que incluye código ejecutable para que pueda ejecutar un proceso aislado en la infraestructura de tecnología de la información (TI). La imagen se compone de bibliotecas del sistema, herramientas del sistema y otras configuraciones de plataformas que un programa de software necesita para ejecutarse en una plataforma de contenerización.

Kernel

Un kernel es el componente central de un sistema operativo. Utilizando la comunicación entre procesos y las llamadas al sistema, actúa como un puente entre las aplicaciones y el procesamiento de datos realizado a nivel de hardware.

Kubernetes

Es una plataforma portátil, extensible y de código abierto para administrar cargas de trabajo y servicios en contenedores, que facilita tanto la configuración declarativa como la automatización. Tiene un ecosistema grande y de rápido crecimiento.

Open Container Iniciatives

Es un proyecto de la Fundación Linux, iniciado en junio de 2015 por Docker para diseñar estándares abiertos para la virtualización a nivel de sistema operativo (contenedores), sobre todo contenedores Linux.

Quay

Es un registro de imágenes de contenedor que le permite crear, organizar, distribuir e implementar contenedores.

Quay.io

Es un registro público alojado que se construye utilizando Quay. (Para ser claros, puede instalar Quay en su infraestructura; Quay.io es una implementación alojada de Quay diseñada para el acceso público).

Red Hat Enterprise Linux

(A menudo abreviado como RHEL) es una distribución de Linux desarrollada por Red Hat para el mercado comercial.

Root

Es el nombre de usuario o cuenta, que de forma predeterminada tiene acceso a todos los comandos y archivos en Linux u otro sistema operativo similar a Unix. También se le conoce como la cuenta raíz, el usuario raíz y el superusuario.

Scripts

Son pequeños programas que no son compilados, es decir, por lo general necesitan de un programa lector o interprete que codifique la información del script y lo lleve a lenguaje de máquina, para que la información sea procesada y ejecutada por el ordenador. Es un término informal que se usa para designar a un programa relativamente simple.